



ARTIFICIAL INTELLIGENCE

Intelligent Agents

ABSTRACT

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

2.1 Introduction

Computers are not very good at knowing what to do: every action a computer performs must be explicitly anticipated, planned for, and coded by a programmer. If a computer program ever encounters a situation that its designer did not anticipate, then the result is not usually pretty—a system crash at best, multiple loss of life at worst.

For the most part, we are happy to accept computers as obedient, literal, unimaginative servants. For many applications (such as payroll processing), it is entirely acceptable. However, for an increasingly large number of applications, we require systems that can decide for themselves what they need to do in order to satisfy their design objectives. Such computer systems are known as **agents**. Agents that must operate robustly in rapidly changing, unpredictable, or open environments, where there is a significant possibility that actions can fail are known as intelligent agents, or sometimes **autonomous agents**.

2.2 Agents

An agent is an entity that perceives its environment (through sensors) and is able to execute actions (through actuator) to change it.

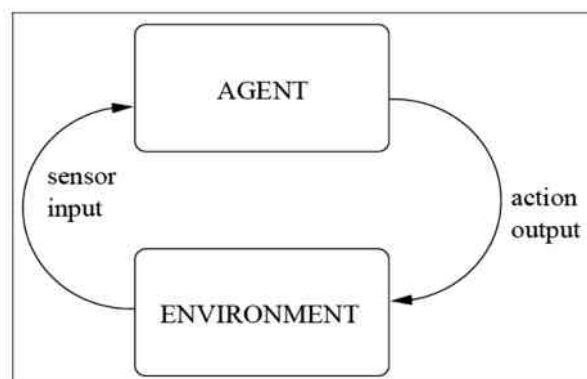


Figure 1.1: An agent in its environment.

Human agent:

- **Sensors:** Eyes (vision), ears (hearing), skin (touch), tongue (gustation), nose (olfaction), neuromuscular system (proprioception)
- **Percepts:** At the lowest level – electrical signals from these sensors
 - After preprocessing – objects in the visual field (location, textures, colors etc.), auditory streams (pitch, loudness, direction) ...

- Actuators/effectors: limbs, digits, eyes, tongue etc.
- Actions: lift a finger, turn left, walk, run, carry an object etc.

Robotic agent:

- Sensors: cameras and infrared range finders for sensors
- Actuators: various motors for actuators

2.3 Agents

An agent's behavior is mathematically described by **agent function**.

The agent function maps from percept histories to actions:

$$[f: P^* \rightarrow A]$$

Practically it is described by an **agent program** which is the real implementation of an agent.

The agent program runs on the physical architecture to produce f

$$agent = architecture + program$$

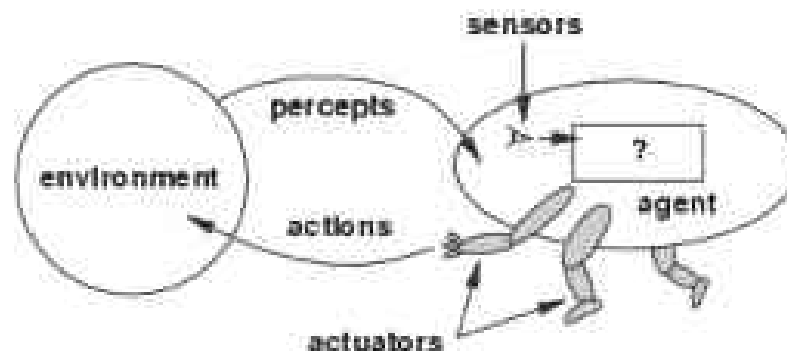


Figure 1.2: An agent in its environment. The agent takes sensory input from the environment, and produces as output actions that affect it. The interaction is usually an ongoing, non-terminating one.

2.3.1 Examples

Any control system can be viewed as an agent. A simple (and overused) example of such a system is a thermostat. Thermostats have a sensor for detecting room or device temperature. The sensed temperature is used to control the device it is attached to, i.e. refrigerator, iron etc.

2.4 Rational agent

- A rational agent is one that does the right thing. So rational agent acts in a way to maximize the achievement of its goals.
- **Performance measure:** An objective criterion for measuring the degree of success of an agent's behavior. E.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.
- For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

2.4.1 Autonomous agents

- An agent is autonomous if its behavior is determined by its own experience (with ability to learn and adapt).
- Therefore, a system is not autonomous if it is guided by its designer according to a priori decisions.
- To survive, agents must have:
 - Enough built-in knowledge to survive.
 - The ability to learn.
- **Example:** Self-driving car: At first, it follows basic rules. But over time, by experiencing different roads, traffic, and obstacles, it learns how to drive better and adapts to new situations without human help.

2.4.2 Omniscience

- An omniscient agent -
 - Knows the actual outcome of its actions in advance
 - Certain that there are no other possible outcomes
 - However, omniscient agent is impossible in real world
- In real situations, an agent can only be rational, not omniscient. Rational agents try to make the best choice based on what they know.

2.5 Designing an intelligent/rational agent: PEAS

PEAS: Performance measure, Environment, Actuators, Sensors

Performance measure:

An objective criterion for measuring the degree of success of an agent's behavior. The performance measure is defined by the designer. E.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.

Environment:

An agent operates in a particular environment. The environment can restrict the behaviors and actions of an agent. Obviously, the more restricted the environment, the easier the agent design is, since the agent has to deal with less factors of the environment.

Actuators:

An actuator is the mechanism by which a control system acts upon an environment. The control system can be simple (a fixed mechanical or electronic system), software-based (e.g. a printer driver, robot control system).

Sensors:

A sensor is a device which detects or measures a physical property and records, indicates, or responds to it.

We must first specify these settings for an intelligent agent design.

Consider, e.g., the task of designing an automated taxi driver:

- What are the performance measure?
- What is the environment?
- What are the actuators?
- What are the sensors?



2.5.1 Automated taxi driving system

Performance measure: Maintain safety, reach destination, minimize costs (fuel, tire wear), obey laws, provide passenger comfort etc.

Environment: Urban streets, freeways, traffic, pedestrians, weather etc.

Actuators: Steer, accelerate, brake, horn, speak / display etc.

Sensors: Video, sonar, speedometer, odometer, engine sensors, keyboard input, microphone, GPS etc.

Different aspects of driving may require different types of agent programs.

2.5.2 More examples

Agent type	Percepts	Actions	Goals	Environment
Medical diagnosis system	Symptoms, findings, patient's interview	Questions, tests, treatments	Healthy patient, minimize costs	Patient, Hospital
Satellite image analysis system	Pixels of varying intensity and color	Print a categorization of scenes	Accurate categorization	Images from orbiting satellite
Part-picking robot	Pixels of varying intensity	Pick up parts and sort into bins	Place parts in correct bins	Conveyor belt with parts
Refinery controller	Temperature, pressure readings	Open, close valves, adjust temperature	Maximize purity, yield, safety	Refinery
Interactive English tutor	Typed words	Print exercises, suggestions, corrections	Maximize students' score on test	Set of students

2.6 Agents programs

- An agent program implements an agent function.
- An agent program takes the current perception $p(t)$ as input.
- An agent program can store the previous perceptions $p(0)$, $p(1)$, ..., $p(t-1)$.
- An agent program gives an action $a(t)$ as output.
- An agent programs is the real implementation of an agent which runs on a physical architecture.



Figure: An agent program outputs an action given a perception

2.6.1 A simple agent program: Table lookup agent

A table-lookup agent is the simplest possible way we can think of to write the agent program. It operates by storing its entire percept sequence, and using it to index into table, which contains the appropriate action for all possible percept sequences.

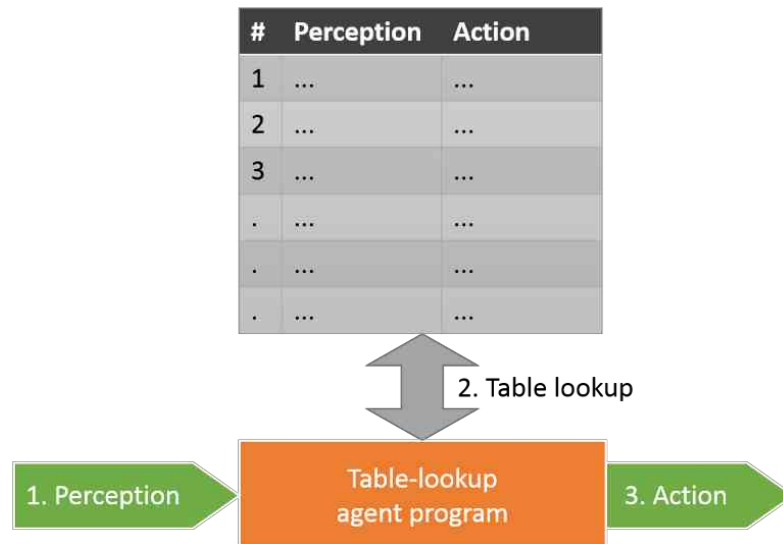


Figure: A simple table-lookup agent program

Drawbacks of table lookup agent program:

While table-lookup agent program is the simplest to implement, it is not realistic. There are some significant drawbacks:

- Requires huge look-up table for complex problems. For example: the table needed for something as simple as an agent that can only play chess would be about 35^{100} entries.
- It would take quite a long time for the designer to build the table.
- The agent has no autonomy at all, because the calculation of best actions is entirely built-in. So if the environment changed in some unexpected way, the agent would be lost.
- Even if we gave the agent a learning mechanism as well, so that it could have a degree of autonomy, it would take forever to learn the right value for all the table entries.

2.6.2 Types of agent Programs

সহায় নোট আছে

Four basic types in order of increasing generality:

- Simple reflex agents
- State/Model-based reflex agents
- Goal-based agents
- Utility-based agents

All these can be turned into learning agents.

2.6.2.1 Simple reflex agents

- A simple reflex agent selects actions based on its current perception of the world and not based on past perceptions. Therefore, no memory is required.
- It works on fully observable environment.
- Every possible perception is connected with an action called condition-action rule which has the form:

If condition then action

- The simple reflex agent has a library of such rules so that if a certain situation arises and it is in the set of Condition-action rules the agent will know how to react with minimal reasoning.

Drawback:

- These agents are simple to work with but have very limited intelligence
- May fall in deadlock or infinite loop.

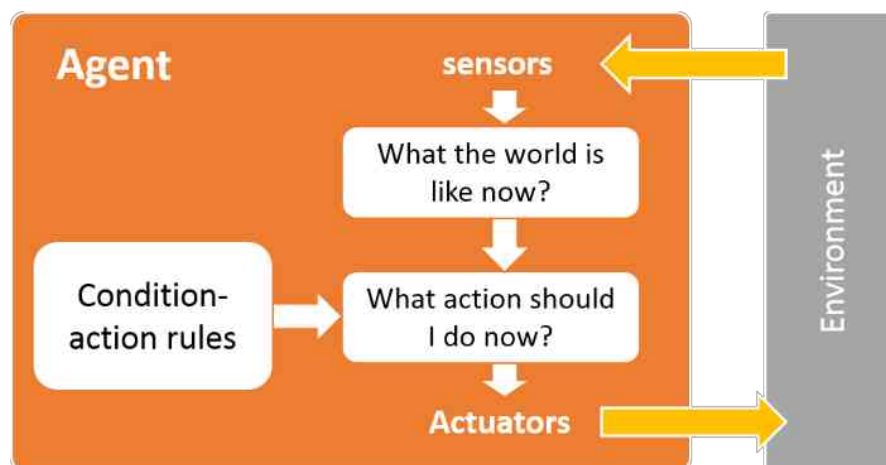


Figure: a simple reflex agent

A **table-lookup agent** is the most basic form of a simple reflex agent, it just stores a huge list of specific responses. A **simple reflex agent** uses **rules** to generalize, so it can handle more inputs with fewer lines of logic.

2.6.2.2 Model based reflex agents

- A model-based reflex agent is one that uses the current perception and its percept history to make decision what action it should select.
- It works on partially observable environment.
- The agent must maintain an internal state that it updates as new sensor information becomes available. Updating the state requires the agent to have two kinds of information:
 - First, it needs information about how the world changes over time (even without doing anything). For example, if it's day now, it'll become night later, this change happens with time.
 - Second, it needs information about how its actions effect the world. For example, if the agent flips a switch, the light turns on, this is the result of its action.

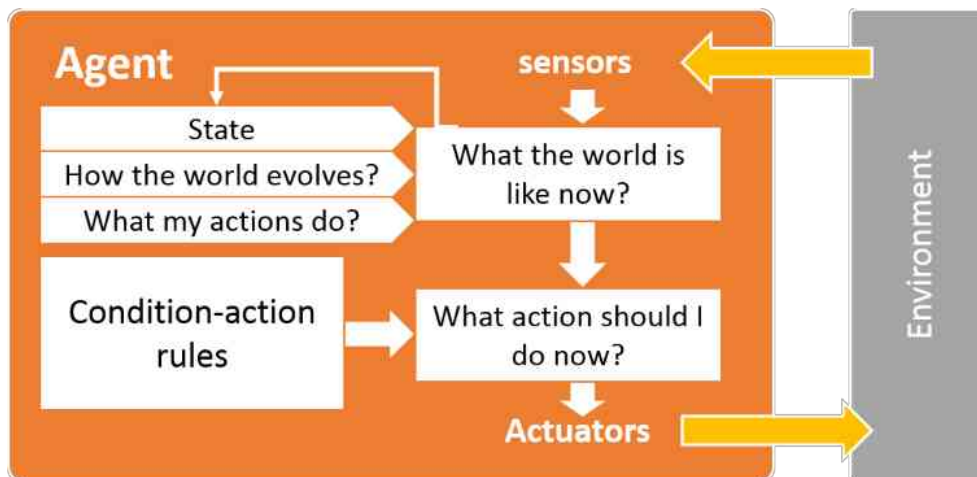


fig: A model based reflex agent

Example: when the brake lights of the car ahead of us come on, the driver has hit the brakes and so the car in front of us is going to slow down. But there's no reason to associate a red light with the deceleration of a vehicle, unless you are used to seeing those two things happen at the same time. So the self-driving-car can learn that it needs to hit the brakes by drawing on its perceptual history.

2.6.2.3 Goal-based agents

- A goal-based agent has a set of goals or desirable situations. The agent can use these goals with a set of actions and their predicted outcomes to see which action sequence is most likely to achieve the goal(s).
- Achieving the goals can take one or more actions. Search and planning are two subfields in AI used to find sequences of actions to achieve an agent's goals.
- Unlike the previous reflex agents before acting this agent reviews many actions and chooses the one which come closest to achieving its goals, whereas the reflex agents just have an automated response for certain situations.

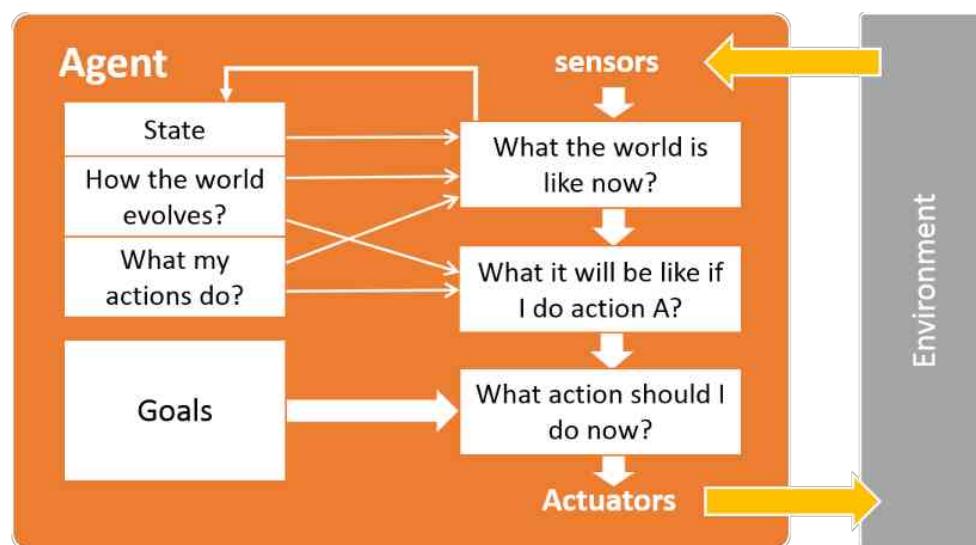


Figure: a goal-based agent

2.6.2.4 Utility-based agents

- These agents are similar to the goal-based agent but provide an extra component of utility measurement which makes them different by providing a measure of success at a given state.
- A goal specifies a crude distinction between a feasible and unfeasible state, but often need a more general performance measure that describes "degree of feasibility"
- Utility function indicates a measure of success or feasibility at a given state.

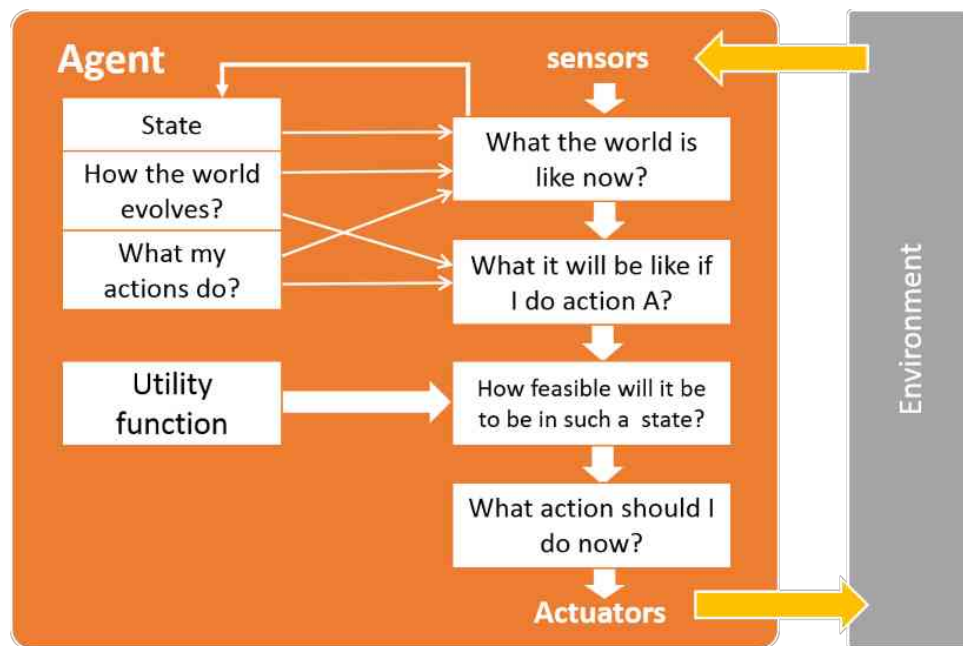


Figure: Utility based agent

Example: Imagine a self-driving car:

- **Goal:** Get to the destination.
- **Utility-Based Agent:** Will consider safety, time, fuel efficiency, and passenger comfort.
- **Action Chosen:** Not just any route, but the route that **maximizes utility** considering all those factors.

2.6.3 Learning agents

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.

A learning agent can be split into the 4 parts described below:

1. **Learning element:** It is responsible for making improvements by learning from environment. Responsible for making improvements by learning.
2. **Critic:** Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard. How well the agent is doing.
3. **Performance element:** It is responsible for selecting external action
4. **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.

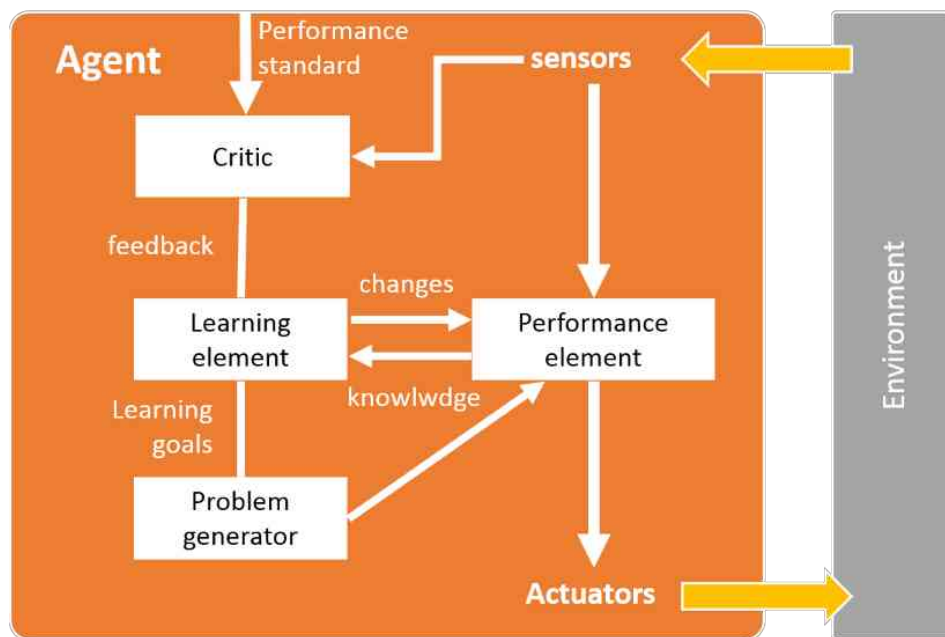


Figure: A learning agent

The learning element is responsible for improvements this can make a change to any of the knowledge components in the agents. One way of learning is to observe pairs of successive states in the percept sequence; from this the agent can learn how the world evolves. For utility based agents an external performance standard is needed to tell the critic if the agent's action has a good or a bad effect on the world.

The performance element is responsible for selecting external actions, and this is considered to be the previous agents discussed.

The learning agent gains feedback from the critic on how well the agent is doing and determines how the performance element should be modified if at all to improve the agent.

The last component is the problem generator. The problem generator only suggests actions that it can already do so we need a way of getting the agent to experience new situations, and this is what the performance generator is for. This way the agent keeps on learning.

Example: Imagine you have a robot that's learning to **clean your room** efficiently. It already knows how to **vacuum**, **pick up toys**, and **move around**.

Now, in AI, such a learning agent has several components:

- **Performance Element** – actually *does* the cleaning.

- **Learning Element** – improves performance over time.
- **Critic** – gives feedback (like “You missed a spot!”).
- **Problem Generator** – suggests new things to try so the agent can learn more.
- **Now Let’s Focus on the Problem Generator & Performance Element**
- The **problem generator** doesn’t invent new actions from scratch—it just says:
“Hey, try doing **this known action** in a **new situation!**”
- So, if the robot only vacuums the same corner of the room every day, it’s not really learning. The **problem generator** might suggest: “Try vacuuming **under the bed** today.”
- Now, that’s a **new experience**, even though the robot already knows how to vacuum. It's trying the same skill in a **new context and that’s how it learns.**

2.7 Environments

2.7.1 Types of environment

FO means the agent can see everything about the environment. Ex- a chess game.

- **Fully observable vs. partially observable**

An environment is fully observable if agent's sensors give it access to the complete state of the environment at each point in time.

PO means it only sees some parts of the environment. Ex-driving a car.

- **Deterministic vs. non-deterministic(stochastic)**

An environment is deterministic if the next state of the environment is only determined by the current state of the environment and the actions selected by the agents. An inaccessible environment might appear to be non-deterministic since the agent has no way of sensing part of the environment and the result of its actions on it. We have to take into consideration the point of view of the agent when determining whether an environment is deterministic or not since the agent might have limited perception capabilities. If the environment is deterministic except for the actions of other agents, then the environment is **strategic**.

- **Episodic vs. non-episodic**

Episodic means actions are independent, each action is a separate episode. Ex- single picture.

In an episodic environment, the agent's experience is divided into episodes which consist of a percept sequence and an action. Since episodes are independent of one another and the agent doesn't need to know the effect of its actions.

In episodic environment, there is a series of one shot actions and only current perception is required for the action.

Non episodic means that past actions affect future ones. Ex- video game.

In **non-episodic (sequential)** environment an agent requires many past actions to determine the next best action.

- **Static vs. dynamic**

An environment is dynamic if it changes while an agent is in the process of responding to a percept sequence. It is static if it does not change while the agent is deciding on an action i.e. the agent does not to keep in touch with time. An environment is semi dynamic if it does not change with time but the agent's performance score does.

static means the environment does not change. Ex- solving math.

dynamic means it can change while the agent thinks. Ex- Busy street

Discrete means a limited number of distinct actions & states. Ex- board game.

continuous means actions and states can vary smoothly. Ex- Driving a car.

- **Discrete vs. continuous**

If the number of percepts and actions in the environment is limited and distinct then the environment is said to be discrete. e.g. A chess board.

- **Single agent vs. multi-agent:** Single agent means one agent operates alone. Ex- vacuum cleaner

An agent operating by itself in an environment.

Multi agent means multiple agents interact in the environment. Ex- multiple cameras.

2.7.2 Examples

Environment	Accessible	Deterministic	Episodic	Static	Discrete
Chess with a clock	Yes	Yes	No	Semi	Yes
Chess w/o a clock	Yes	Yes	No	Yes	Yes
Taxi driving	No	No	No	No	No
Medical diagnosis system	No	No	No	No	No
Image analysis system	Yes	Yes	Yes	Semi	No
Part-picking robot	No	No	Yes	No	No